GYM MANAGEMENT SYSTEM

A Major Project Report Submitted
In partial fulfillment of the requirements for the
award of the degree of

BACHELOR OF TECHNOLOGY
in
INFORMATION TECHNOLOGY

by

Aditya kumar sahoo	B424003
Adityanarayan sahu	B424006
Ajit behera	B424007
Asutosh dalai	B424017

Under the guidance of

Utkalika Satapathy

Assistant Professor
Dept. of CSE
IIIT Bhubaneswar,India

International Institute of Information Technology,Bhubaneswar
Department of Computer Science & Engineering
UGC Autonomous Institute, Govt. of India
(Established by Govt. of Odisha)
Bhubaneswar – 751013, Odisha, India
www.iiit-bh.ac.in

2024– 2028

International Institute of Information Technology,Bhubaneswar
Department of Computer Science & Engineering
UGC Autonomous Institute, Govt. of India
(Established by Govt. of Odisha)
Bhubaneswar – 751013, Odisha, India

CERTIFICATE

This is to certify that the Major Project work entitled “GYM MANAGEMENT SYSTEM” is carried out

by Group Query Crafters, in partial fulfillment for the award of degree in

Bachelor of Technology in INFORMATION TECHNOLOGY,

International Institute of Information Technology,Bhubaneswar

during the academic year 2024-2028.

Internal Guide

Utkalika Satapathy
Assistant Professor
Dept. of CSE

External Examiner

DECLARATION

I hereby declare that the project titled "QUERY CRAFTERS" submitted to International Institute of Information Technology, Bhubaneswar, Department of INFORMATION TECHNOLOGY (UGC-Autonomous), affiliated for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a result of original research carried-out in this thesis. I understand that my report may be made electronically available to the public. It is further declared that the project report or any part there of has not been previously submitted to any University.

Group:Query Crafters

Degree : Bachelor of Technology in INFORMATION TECHNOLOGY

Department : INFORMATION TECHNOLOGY

Title of the project :GYM MANAGEMENT SYSTEM

Acknowledgement

We would like to express our deepest gratitude to our project advisor, Prof.Utakalika satapathy, for her guidance, support, and patience throughout the entire process of reproject design. Without her insightful feedback and encouragement, this work would not have been possible.

We would also like to thank my fellow-mates, for their valuable comments and suggestions, which helped to shape and refine this thesis.

Finally, I would like to express my gratitude to internal guide, without whom this research would not have been possible. Their willingness to share their experiences and insights has enriched this work and contributed to the advancement of knowledge in this field.

Thank you all for your invaluable contributions to this project.

ABSTRACT

This project presents the design and development of a **Gym Management System** built using a modern full-stack architecture. The system enables efficient management of gym operations such as member registration, trainer assignment, workout scheduling, attendance tracking, and payment handling through a user-friendly web interface. User interactions are processed through integrated backend functionalities, ensuring smooth execution of business logic and seamless communication between system components.

Data is effectively managed using a relational database system, ensuring structured storage and retrieval of information related to members, trainers, subscriptions, and fitness progress. The backend is developed using a robust server-side framework, which handles data processing, validation, and API communication, while the frontend provides an interactive and responsive user experience.

The application incorporates a secure authentication mechanism with role-based access control, allowing different user types— Trainer, and Member—to access specific functionalities. Members can track their fitness progress, including BMI and workout routines, trainers can monitor and guide assigned members, and administrators can oversee system operations, payments, and reports.

Overall, the system provides an efficient, secure, and scalable solution for managing gym activities. It enhances operational efficiency, reduces manual effort, and improves user experience through automation and centralized data management.

TABLE OF CONTENTS

1. INTRODUCTION

2. SYSTEM OVERVIEW

3. SYSTEM ARCHITECTURE

4. MODULE DESIGN

5. DATABASE DESIGN

6. SECURITY DESIGN

7. Entity-Relationship (ER) Diagram

8. ASSUMPTIONS

9. CONCLUSIONS

1. Introduction

1.1 Purpose

This Software Design Document (SDD) describes the complete software design of the Gym Membership and Workout Tracking System. It provides detailed information about the system architecture, module breakdown, data flow, database schema, and security considerations to serve as a blueprint for development and academic evaluation.

1.2 Scope

The system manages all core gym operations including member registration, trainer management, membership plan administration, workout assignment, daily attendance tracking, and payment processing. It is designed as a relational database-backed application suitable for single-gym environments.

1.3 Definitions

Term	Definition
DBMS	Database Management System — software for storing and managing structured data
UI	User Interface — the visual front-end through which users interact with the system
DDL	Data Definition Language — SQL commands for defining database structure (CREATE, ALTER, DROP)
DML	Data Manipulation Language — SQL commands for manipulating data (INSERT, UPDATE, DELETE, SELECT)
ER	Entity-Relationship — a modeling technique used to design the database schema
PK	Primary Key — a unique identifier for each record in a table
FK	Foreign Key — a field that links one table to another

2. System Overview

The Gym Membership and Workout Tracking System is a database-driven application that enables two categories of users — Trainers, and Members — to interact with gym-related data efficiently.

User Role	Primary Capabilities
Trainer	View assigned members, manage workout plans, View Profile and Total Earnings
Member	View profile, track attendance, view payment status, view all workout plans, view membership plan expiry date

3. System Architecture

The system follows a classic 3-Tier Architecture ensuring clear separation of concerns:

Layer	Component	Responsibility
Presentation Layer	User Interface (Web/Desktop)	Forms, reports, dashboards for user interaction
Application Layer	Business Logic Module	Validation, processing, role-based access control
Database Layer	Relational DBMS (MySQL)	Persistent storage, query processing, integrity

This separation ensures maintainability — changes to the UI do not affect business logic or the database schema, and vice versa.

4. Module Design

4.1 Member Module

- Register new members with personal details, contact, joining date, and status
- Secure login using Phone no. and password
- View and update personal profile
- Enrol in available membership plans
- View workout plans, Attendance, payment History, attendance history, membership expiry date.

4.2 Trainer Module

- Register new Trainers with personal details, contact, and specialisation.
- Secure login using Phone no. and password
- Assign trainers to members based on plan and availability
- View list of assigned members
- Assign workout plans to assigned trainees.
- View Total Earnings
- View Profile

4.3 Membership Module

- View and membership plans
- Update plan pricing and duration
- Link plans to members upon enrolment

4.4 Workout Module

- Define workout plans with exercises
- Assign workout plans to members by trainer
- Track member progress over time

4.5 Attendance Module

- Record daily check-in and check-out for each member

4.6 Payment Module

- View Payment History

5. Database Design

The system uses a Relational Database Management System. Below are the primary tables and their key fields:

Table	Primary Key	Key Attributes
Member	member_id	name, age, contact, joining_date, status, plan_id (FK), trainer_id (FK)
Trainer	trainer_id	name, specialization, contact, experience_years
Membership_Plan	plan_id	plan_name, duration_months, price, description
Workout_Plan	workout_id	plan_name, trainer_id (FK), exercises, frequency
Attendance	attendance_id	member_id (FK), date, check_in, check_out, status
Payment	payment_id	member_id (FK), amount, payment_date, due_date, mode, status

6. Security Design

6.1 Security Design

- User Authentication: Login with Member ID / Trainer ID and password
- Role-Based Access Control: Trainers and Members see only their own data

7. Entity-Relationship (ER) Diagram

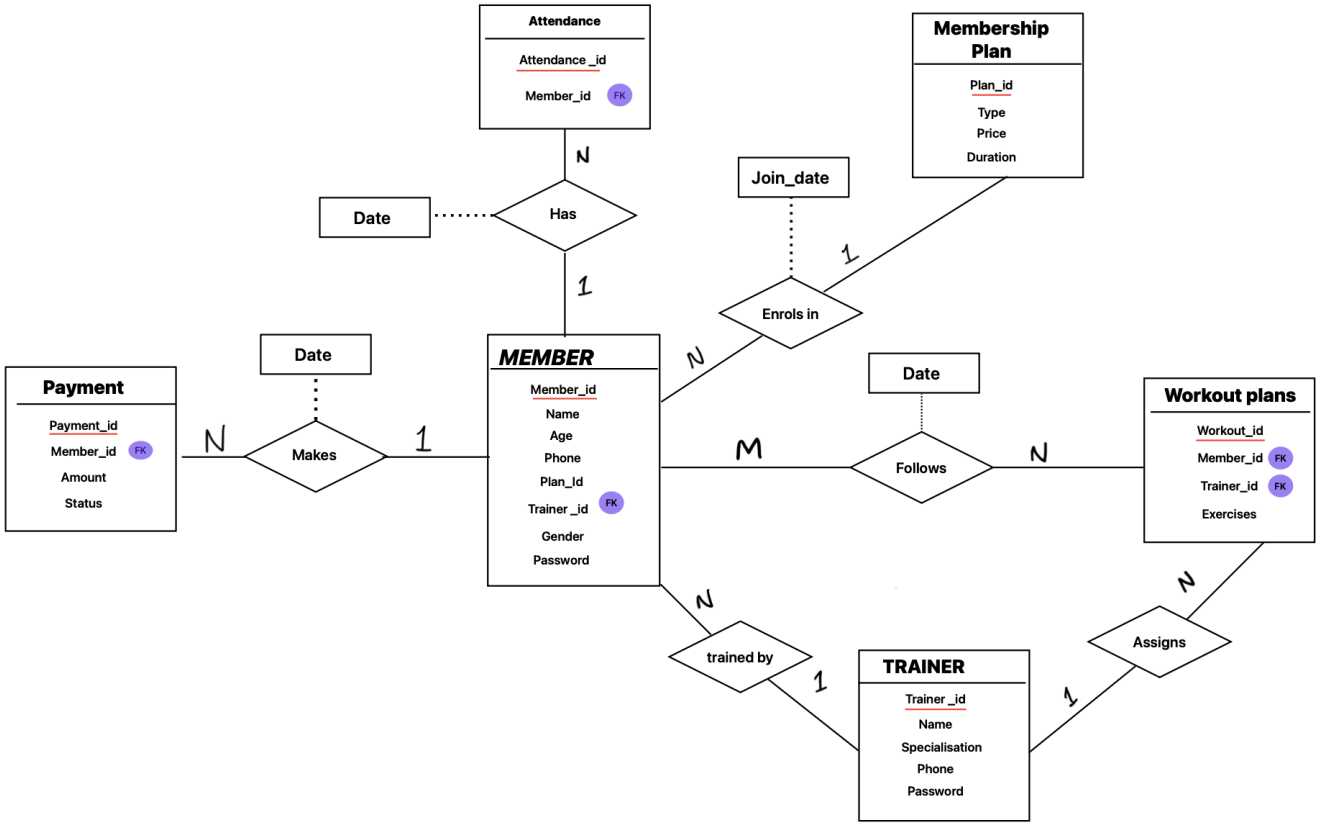
7.1 Entities & Relationships

Relationship	Cardinality	Description
Member — Membership Plan	Many-to-One (M:1)	Many members can subscribe to one plan
Member — Trainer	Many-to-One (M:1)	Many members can be assigned to one trainer
Member — Attendance	One-to-Many (1:M)	One member has many attendance records
Member — Payment	One-to-Many (1:M)	One member has many payment transactions
Trainer — Workout Plan	One-to-Many (1:M)	One trainer can create many workout plans
Member—Workout Plan	Man-to-Many (1:M)	One Member can create many workout plans

7.2 Entity Attributes Summary

Entity	Attributes
payment	payment_id, member_id, amount,date,status
trainer	trainer_id (PK), name, specialization, phone, password
membershipplan	plan_id (PK), duration, price, type
workoutplan	workout_id (PK), member_id, trainer_id (FK), exercises, date
attendance	attendance_id (PK), member_id (FK), date
payment	payment_id (PK), member_id (FK), amount, payment_date, due_date, mode, status
member	member_id, name, age, gender, phone, join_date, plan_id, trainer_id, password

ER DIAGRAM



Schema design

1. **MEMBER (member_id PK)**

├── plan_id → MembershipPlan(plan_id)

├── trainer_id → Trainer(trainer_id)

```
mysql> desc member;
+------------+--------------+------+-----+---------+----------------+
| Field      | Type         | Null | Key | Default | Extra          |
+------------+--------------+------+-----+---------+----------------+
| member_id  | int          | NO   | PRI | NULL    | auto_increment |
| name       | varchar(100) | YES  |     | NULL    |                |
| age        | int          | YES  |     | NULL    |                |
| gender     | varchar(10)  | YES  |     | NULL    |                |
| phone      | varchar(15)  | YES  |     | NULL    |                |
| join_date  | date         | YES  |     | NULL    |                |
| plan_id    | int          | YES  | MUL | NULL    |                |
| trainer_id | int          | YES  | MUL | NULL    |                |
| password   | varchar(100) | YES  |     | NULL    |                |
+------------+--------------+------+-----+---------+----------------+
9 rows in set (0.09 sec)
```

2. **TRAINER (trainer_id PK)**

```
mysql> desc trainer;
+----------------+--------------+------+-----+---------+----------------+
| Field          | Type         | Null | Key | Default | Extra          |
+----------------+--------------+------+-----+---------+----------------+
| trainer_id     | int          | NO   | PRI | NULL    | auto_increment |
| name           | varchar(100) | YES  |     | NULL    |                |
| specialization | varchar(100) | YES  |     | NULL    |                |
| phone          | int          | YES  |     | NULL    |                |
| password       | varchar(20)  | YES  |     | NULL    |                |
+----------------+--------------+------+-----+---------+----------------+
5 rows in set (0.00 sec)
```

3. **WORKOUTPLAN (workout_id PK)**

├── member_id → Member(member_id)

├── trainer_id → Trainer(trainer_id)

```
mysql> desc workoutplan;
+------------+------+------+-----+---------+----------------+
| Field      | Type | Null | Key | Default | Extra          |
+------------+------+------+-----+---------+----------------+
| workout_id | int  | NO   | PRI | NULL    | auto_increment |
| member_id  | int  | YES  | MUL | NULL    |                |
| trainer_id | int  | YES  | MUL | NULL    |                |
| exercises  | text | YES  |     | NULL    |                |
| date       | date | YES  |     | NULL    |                |
+------------+------+------+-----+---------+----------------+
5 rows in set (0.00 sec)
```

4. **PAYMENT (payment_id PK)**

├── member_id → Member(member_id)

```
mysql> desc payment;
+------------+---------------+------+-----+---------+----------------+
| Field      | Type          | Null | Key | Default | Extra          |
+------------+---------------+------+-----+---------+----------------+
| payment_id | int           | NO   | PRI | NULL    | auto_increment |
| member_id  | int           | YES  | MUL | NULL    |                |
| amount     | decimal(10,2) | YES  |     | NULL    |                |
| date       | date          | YES  |     | NULL    |                |
| status     | varchar(20)   | YES  |     | NULL    |                |
+------------+---------------+------+-----+---------+----------------+
5 rows in set (0.00 sec)
```

5. **ATTENDANCE (attendance_id PK)**

├── member_id → Member(member_id)

```
mysql> desc attendance;
+---------------+------+------+-----+---------+----------------+
| Field         | Type | Null | Key | Default | Extra          |
+---------------+------+------+-----+---------+----------------+
| attendance_id | int  | NO   | PRI | NULL    | auto_increment |
| member_id     | int  | YES  | MUL | NULL    |                |
| date          | date | YES  |     | NULL    |                |
+---------------+------+------+-----+---------+----------------+
```

6. **MEMBERSHIPPLAN (plan_id PK)**

```
mysql> desc membershipplan;
+----------+---------------+------+-----+---------+----------------+
| Field    | Type          | Null | Key | Default | Extra          |
+----------+---------------+------+-----+---------+----------------+
| plan_id  | int           | NO   | PRI | NULL    | auto_increment |
| type     | varchar(50)   | YES  |     | NULL    |                |
| price    | decimal(10,2) | YES  |     | NULL    |                |
| duration | int           | YES  |     | NULL    |                |
+----------+---------------+------+-----+---------+----------------+
4 rows in set (0.00 sec)
```

The database consists of six main tables: Member, Trainer, WorkoutPlan, MembershipPlan, Payment, and Attendance. These tables are interconnected using primary and foreign key relationships to ensure data integrity.

8. Assumptions

- Users have basic familiarity with computer systems and form-based interfaces
- Internet connectivity is available if the system is deployed as a web application
- Member IDs and Trainer IDs are assigned sequentially
- Each member is enrolled in exactly one active membership plan at a time

9. Conclusion

This Software Design Document provides a comprehensive blueprint for the Gym Membership and Workout Tracking System. The project successfully demonstrates the practical application of Database Management System concepts in a real-world scenario.

Through this project, the following objectives have been achieved:

- Designed a normalized relational database schema with proper primary and foreign key relationships
- Modelled the system using Entity-Relationship diagrams
- Implemented SQL queries covering DDL and DML operations
- Applied a 3-tier architecture ensuring clean separation of UI, logic, and data
- Incorporated security measures including role-based access control and input validation

The system effectively improves data accuracy, reduces redundancy, and provides a structured approach to managing gym operations. This project demonstrates the power of DBMS in digital zing and streamlining real-world administrative tasks.

References

- **Official documentation of Next.js**

https://nextjs.org/docs

- **Official documentation of React**

https://react.dev/

- **Official documentation of Tailwind CSS**

https://tailwindcss.com/docs

- **Official documentation of PostgreSQL**

https://www.postgresql.org/docs/

- **Official documentation of Drizzle ORM**

https://orm.drizzle.team/docs/overview

- **Official documentation of Zustand**

https://docs.pmnd.rs/zustand/getting-started/introduction

- **Official documentation of jose**

https://github.com/panva/jose

- **Official documentation of Bun**

https://bun.sh/docs

- **Official documentation of Turbopack**

https://turbo.build/pack/docs

- **Visual Studio Code Official Site**

https://code.visualstudio.com/docs